

面向内存受限环境的大语言模型推理内存管理优化系统

法尔孔队：李子恒，邓晓川，蔡梓涵

赛题编号：proj59

Abstract

大语言模型本地推理需要同时管理模型权重、KV cache 和 MoE expert 参数。在物理内存受限环境中，按需映射的模型权重会在首次访问和 expert 激活阶段引入大量 major page faults，进而放大 decode 延迟和尾部抖动。本文围绕赛题给出的三个任务组织系统设计：第一，分析 LLM 推理过程中的参数加载、KV cache 使用和专家激活访存行为；第二，使用虚拟内存、按需加载和替换策略模拟降低运行时物理内存压力；第三，使用预取与异步调度覆盖 I/O 延迟、提升推理性能。本文系统基于 llama.cpp 扩展 tensor、KV、expert 和 memory trace，利用 MoE routing 语义触发 madvise/posix_fadvise 类型 OS hint，并通过用户态异步队列和 deadline_score 调度将 hint 操作从 decode 关键路径迁出。基于本地 Qwen3.5-35B-A3B GGUF 模型的 N=3 重复实验表明，相比 baseline，推荐策略 deadline_score 将 decode 均值从 376069 us 降至 194971 us，major faults 均值从 757052 降至 71060，swap 均值从 217.86 MiB 降至 123.67 MiB。实验说明，结合模型结构语义的用户态预取调度能够有效服务赛题的访存分析、内存压力控制和 I/O 延迟覆盖目标。
关键词：大语言模型推理；内存管理；MoE；页面预取；OS hint；llama.cpp

1 引言

大语言模型（Large Language Model, LLM）的推理部署正在从云端高配服务器扩展到开发机、边缘设备和其他内存受限场景。与传统服务端推理不同，受限环境中的主要瓶颈不仅来自计算吞吐，还来自模型权重映射、KV cache 增长、MoE expert 选择性激活和操作系统页面回收之间的复杂交互。当模型权重以 mmap 形式按需加载时，首次触碰权重页会触发 page faults；当物理内存不足时，major faults、swap 增长和页缓存竞争会进一步影响 decode 延迟。

现有本地推理框架 llama.cpp 已经提供了高效、可移植的 LLM 推理能力，但其默认运行路径缺少面向操作系统内存行为的细粒度 trace，也无法直接利用 MoE routing 语义向内核暴露未来访问信息。另一方面，直接修改内核页替换策略虽然可能取得更底层的控制能力，但会提高部署门槛和复现实验成本。本文工作的核心问题是：在保持用户态低侵入实现的前提下，能否利用 LLM 推理过程中可见的模型结构和 routing 信息改善内存受限推理的页面行为？

本文提出并实现 LLM 推理内存管理优化系统。系统以 llama.cpp 为基础，围绕 tensor load、first touch、KV cache、expert routing 和 RSS/swap 指标建立 trace pipeline，并在此基础上设计默认关闭、显式开启的

OS hint 实验路径。对于 MoE 模型，系统将 routed expert 映射为 expert slice，以 (layer, expert, tensor) 为粒度进行预取、统计和离线模拟。为避免同步 hint 调用阻塞 decode 关键路径，系统进一步实现用户态异步队列，并使用 deadline_score 优先级综合考虑 expert 即将被使用的紧迫度和 router score。

本文贡献如下：

- 构建了面向 LLM 推理的 tensor/KV/expert/memory trace 与自动分析 pipeline，可输出单次报告、多运行对比和重复实验摘要。
- 实现了基于 MoE routing 的 expert-aware OS hint 机制，在用户态通过 madvise(MADV_WILLNEED) 与 posix_fadvise(POSIX_FADV_WILLNEED) 影响页面预取。
- 设计了异步 expert prefetch 队列和 deadline-aware priority 调度，将大量 hint 调用从 decode 路径迁移到后台 worker。
- 通过 trace-driven 模拟比较 route、LRU、LFU、window-LFU 和 least-stale 等替换策略，并用 N=3 重复实验评估 latency、major faults、RSS、swap 和 hint calls 的综合权衡。

2 赛题任务与总体路线

赛题要求面向嵌入式设备、边缘计算设备等内存受限环境部署大模型。大模型参数规模和推理时 KV cache 的持续增长使资源受限场景难以直接部署；同时，LLM 推理访存具有较强结构性，MoE 模型每个 token 仅激活少量 expert，这为虚拟内存、按需加载、数据预取和数据换出提供了系统优化空间。

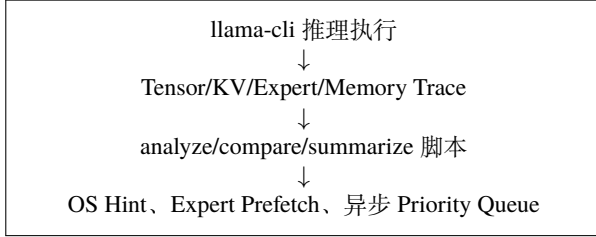
本文将工作拆解为三条与赛题任务直接对应的技术路线：

1. **访存行为分析。**建立 tensor、KV、expert 和 memory trace，分析参数加载、KV cache 使用、专家激活和 page fault 分布。
2. **虚拟内存与内存压力控制。**在不修改内核的前提下使用 mmap、OS hint、expert slice cache 抽象和离线替换策略模拟，评估按需加载、数据换出和缓存预算对物理内存压力的影响。
3. **预取覆盖 I/O 延迟。**将 MoE routing 语义转化为未来访问信号，结合 expert-aware prefetch、异步 hint queue 和 deadline-aware priority 调度降低 major faults 和 decode 延迟。

本文选择用户态 OS hint 而非内核修改作为初赛阶段主线，原因包括：第一，madvise 和 posix_fadvise

Table 1: 赛题任务与本文实现对应关系

赛题任务	本项目实现
访存行为分析	tensor/KV/expert/memory JSONL trace; <code>analyze_trace.py</code> 聚合参数加载、KV 增长、expert routing、RSS、swap 和 faults。
虚拟内存与数据换出	mmap 权重访问; expert slice cache; LRU/LFU/window-LFU/least-stale 离线模拟; route TTL 和 hint summary。
预取覆盖 I/O 延迟	route-based expert prefetch; <code>madvice/poix_fadvise</code> ; 异步队列; <code>deadline_score</code> priority。

**Figure 1:** 系统总体结构示意图。正式论文版本可替换为由 trace pipeline 生成的架构图或指标图。**Table 2:** 核心模块与职责

模块	职责
<code>tensor_trace.cpp</code>	记录 tensor load/first touch, 执行可选 OS hint 和 expert prefetch。
<code>expert_trace.cpp</code>	记录 MoE routing、layer/expert 激活和 expert 访问事件。
<code>kv_trace.cpp</code>	记录 KV cache append/reuse, 用于长上下文内存增长分析。
<code>analyze_trace.py</code>	聚合 trace 指标并生成图表、HTML 和 Markdown 报告。
<code>simulate_expert_cache.py</code>	离线比较 expert cache 替换策略。
<code>run_trace_pipeline.sh</code>	自动执行单次推理、收集 trace 并生成分析结果。

可直接在普通 Linux 环境复现；第二，用户态能够获得 tensor 名称、层号、expert id、router score 等内核不可见信息；第三，优化路径可以通过环境变量显式开启，保证 baseline 行为不被默认改变。

3 系统设计

图 1 给出了系统总体结构。推理阶段由 llama-cli 执行模型加载与 token 生成；插桩模块将 tensor、KV、expert 和 memory 事件写入 JSONL trace；分析脚本聚合 page faults、RSS、swap、hint events、KV 增长和 expert 行为；优化模块根据环境变量触发 OS hint、expert prefetch、异步队列和优先级调度。

3.1 Trace 数据流

系统运行时将所有实验输出放置在 `llama.cpp/trace_output/<RUN_NAME>/` 下。单次 pipeline 的主要步骤包括：设置 trace 输出目录，执行 llama-cli，收集标准

输出、标准错误和 JSONL 事件，随后调用 `analyze_trace.py` 生成 `metrics.json`、图表和 HTML 报告。跨运行比较由 `compare_trace_runs.py` 完成，重复实验聚合由 `summarize_repeat_runs.py` 完成。

3.2 Expert slice 抽象

系统将 MoE expert 权重抽象为 expert slice，并以 $(layer, expert, tensor)$ 作为逻辑缓存项。每个条目维护 size、last step、hit count、score、resident/advised 等状态。该粒度避免将不同层的同名 expert 混为一谈，适合表达 layer-local 热点和 route 触发的未来访问集合。

4 面向三项任务的关键实现

4.1 任务一：推理访存行为分析

系统在 `llama.cpp/trace/` 中扩展 trace sink，将推理期间的 tensor、KV、expert 和 memory 事件写入 JSONL。分析脚本进一步聚合模型加载、first touch、KV cache append/reuse、expert routing、RSS、swap 和 page faults。该链路解决的是“看清楚内存行为”的问题：它不仅记录是否发生缺页，还能把缺页与 tensor 名称、层号、expert id 和推理阶段关联起来。

KV cache 分析统计 K/V 各自占用、PREFILL/DECODE 阶段占用、每 1k tokens 增长和 2k/4k/8k 上下文投影，用于评估长上下文下内存增长趋势。Expert trace 则记录 MoE routing 分布，为后续 route-based prefetch 和 expert slice cache 提供访问序列。

4.2 任务二：虚拟内存与数据换出策略

系统将 expert 权重抽象为 expert slice，并以 $(layer, expert, tensor)$ 作为逻辑缓存项。每个条目维护 size、last step、hit count、score、resident/advised 等状态。该粒度避免将不同层的同名 expert 混为一谈，适合表达 layer-local 热点和 route 触发的未来访问集合。

为避免盲目在真实推理中试错，系统实现 trace-driven expert cache 模拟，用于比较 route、lru、lfu、window_lfu 和 least_stale。模拟覆盖 128/256/512/768/1024 MiB 等预算。当前 trace 下，朴素 LRU/LFU 类策略在不超过 1 GiB expert slice budget 时 miss 与 eviction 过高，难以替代完整 route prefetch。因此真实运行候选保留 route coverage，并用 TTL、summary 和异步调度减少重复 hint 与关键路径阻塞。

4.3 任务三：预取覆盖 I/O 延迟

最直接的优化是根据当前 token 的 routed experts 对相应 expert slice 执行预取。系统通过 `LLM_MEM_TRACE_OPT_EXPERT_PREFETCH=1` 开启该路径，并使用 `LLM_MEM_TRACE_OPT_EXPERT_POLICY=route` 选择 route 策略。对于 mmap tensor，系统对 page-aligned 虚拟地址范围执行 `madvice(MADV_WILLNEED)`；对于可定位文件映射的范围，可进一步执行 `posix_fadvise(POSIX_FADV_WILLNEED)`。

完整 route prefetch 能保持较高 coverage，但同步 hint 调用数量接近十万级。如果这些调用发生在 decode 路径上，syscall 开销会抵消一部分 page fault 改善收益。因此，后续优化重点从简单减少 expert 数量转向降低同步阻塞。

4.4 异步 Hint Queue 与 Priority

异步 expert prefetch 由 LLM_MEM_TRACE_OPT_EXPERT_ASYNC=1 开启。推理线程作为 producer 将 ExpertHintTask 放入队列，后台 worker 执行实际 madvise/posix_fadvise。当队列满或 worker 不可用时，系统回退到同步路径，保证功能可用。进程退出时写出 EXPERT_ASYNC_SUMMARY，记录 enqueue、issued、fallback 和队列深度等信息。

FIFO 异步队列无法区分任务紧迫度，可能优先处理距离使用点较远的 expert slice。系统实现三种 priority mode: score、deadline 和 deadline_score。其中 deadline_score 先按照 step/layer 接近程度排序，再用 router score 打破同等紧迫度下的优先级。该策略对应当前主推荐配置 route_all_async4_deadline_score。

5 实验设计

5.1 实验环境与模型

实验基于 Linux 环境、CMake 构建系统和开启 LLAMA_MEM_TRACE=ON 的 llama-cli。默认模型路径为 models/Qwen3.5-35B-A3B-Q3_K_M.gguf，复现时可通过 MODE_L_FILE 指定其他 GGUF 模型。由于模型文件较大，仓库不提交权重文件。

5.2 对比策略

本文初版纳入四个重复实验组：

- **baseline**: 关闭 OS hint，作为默认推理行为对照。
- **expert_prefetch**: 同步 route-based expert prefetch，强调 major faults 改善。
- **deadline_score**: 异步队列、4 个 worker、deadline-aware priority，是当前综合推荐策略。
- **decode_ttl1**: 引入 route hint TTL，作为降低 hint calls 的 Pareto 备选。

每组执行 N=3 重复实验，统计 decode latency、major faults、RSS、swap 和 hint calls。baseline 延迟波动较大，因此本文使用重复实验均值作为主要比较依据。

6 结果分析与可视化

表 3 给出了 N=3 重复实验的原始均值和标准差。为突出赛题三项任务上的收益，图 2 进一步展示相对 baseline 的归一化效果。三个指标均以 baseline 为 1.0，数值越低代表消耗越低。

从任务角度看，trace pipeline 覆盖了参数加载、KV cache、expert routing、RSS/swap 和 page faults，支撑了赛题第一项访存行为分析；expert slice cache、替换策略模拟和 TTL 控制服务于第二项虚拟内存与数据换出；route-based prefetch、异步 hint queue 和 deadline-aware priority 则对应第三项预取覆盖 I/O 延迟。三项任务的最终效果共同体现在 decode、major faults 和 swap 的下降上。

deadline_score 是当前综合推荐策略。相对 baseline，它将 decode 均值从 376069 us 降至 194971 us，降低约 48.2%；将 major faults 从 757052 降至 71060，降低

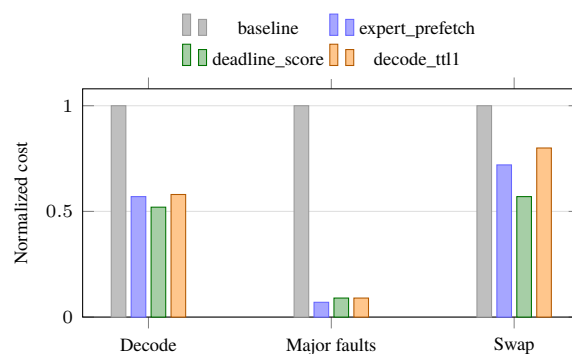


Figure 2: 相对 baseline 的归一化指标对比。数值越低越好；deadline_score 在 decode 和 swap 上最优，expert_prefetch 在 major faults 上最优。

约 90.6%；将 swap 从 217.86 MiB 降至 123.67 MiB，降低约 43.2%。RSS 从 6.228 GiB 小幅增加到 6.433 GiB，说明主要收益并非来自压低常驻集，而是来自更及时的页面预取和更少的 swap 压力。

expert_prefetch 的 major faults 最低，证明完整 route coverage 对 I/O 延迟覆盖有效；但同步 hint 调用仍会干扰 decode 路径。decode_ttl1 的 hint calls 最低，是减少重复 hint 的备选，但 latency、RSS 和 swap 不如主策略。离线替换策略模拟也表明，在 1 GiB 以内的 expert slice cache budget 下，LRU/LFU/window-LFU/least-stale 等策略 miss 与 eviction 过高，无法稳定替代 route 语义。

7 可复现性

系统提供从构建到重复实验的完整脚本。核心构建命令如下：

```
cmake -S llama.cpp -B llama.cpp/build \
  -DLLAMA_MEM_TRACE=ON -DCMAKE_BUILD_TYPE=Release
cmake --build llama.cpp/build --target llama-cli -j"${nproc}"
```

单次 baseline 可通过 run_trace_pipeline.sh 执行：

```
MODEL_FILE=/path/to/model.gguf \
RUN_NAME=baseline \
NUM_TOKENS_PREDICT=80 \
LLM_MEM_TRACE_OS_HINTS=0 \
bash llama.cpp/trace/run_trace_pipeline.sh
```

最终重复实验矩阵由 run_finalist_repeat_matrix.sh 固化，默认 dry-run，设置 RUN_REPEAT_MATRIX_EXECUTE=1 后执行真实 N 次实验：

```
RUN_REPEAT_MATRIX_EXECUTE=1 \
RUN_PREFIX=contest_finalist \
REPEAT_COUNT=3 \
MODEL_FILE=/path/to/model.gguf \
bash llama.cpp/trace/run_finalist_repeat_matrix.sh
```

脚本输出目录为 llama.cpp/trace_output/，该目录已从提交中排除。

仓库还提供 Python 和 shell 静态检查命令，包括 py_compile、bash -n，用于在不执行长时间推理的情况下检查脚本和实验入口的基本可用性。

```
python3 -m py_compile \
  llama.cpp/trace/analyze_trace.py \
  llama.cpp/trace/compare_trace_runs.py \
  llama.cpp/trace/simulate_expert_cache.py \
  llama.cpp/trace/summarize_repeat_runs.py

bash -n llama.cpp/trace/run_trace_pipeline.sh
bash -n llama.cpp/trace/run_finalist_repeat_matrix.sh
```

Table 3: N=3 重复实验结果

方案	Decode 均值/us	Decode std	Major faults 均值	RSS 均值/GiB	Swap 均值/MiB	Hint calls 均值
baseline	376069	92579	757052	6.228	217.86	0
expert_prefetch	214724	17077	49613	6.486	157.44	99907
deadline_score	194971	12390	71060	6.433	123.67	98893
decode_ttl	216819	15777	69406	6.501	174.25	82711

8 局限性与后续工作

当前结果仍有三个限制。第一，实验基于本地单一模型、机器和 prompt，泛化性需要在更多模型大小、内存压力和上下文长度下验证。第二，hint calls 仍接近十万级，虽然异步化降低了 decode 干扰，但 syscall 数量本身仍有优化空间。第三，KV cache 当前主要完成分析和投影，尚未实现类似 PagedAttention/vAttention 的分页式 KV allocator。

后续工作将围绕三个方向推进：其一，探索更精确的 range batching 和 expert slice 合并，减少 syscall 数量而不损害 route coverage；其二，在多模型、多硬件环境下重复验证 deadline_score 的稳定性；其三，将 KV cache trace 与 allocator 设计结合，进一步覆盖长上下文场景下的内存增长问题。

9 结论

本文提出的 LLM 推理内存管理优化系统展示了一条低侵入、可复现的内存受限 LLM 推理优化路径。系统不修改内核，而是利用 llama.cpp 中可见的 tensor 与 MoE routing 语义，在用户态完成 trace、OS hint、expert prefetch、异步队列和重复实验评估。实验表明，deadline_score 策略能够在小幅增加 RSS 的情况下显著降低 decode latency、major faults 和 swap，说明模型语义感知的页面预取调度是受限内存环境下 LLM 推理优化的有效方向。

References

- [1] Gerganov et al. *llama.cpp*: LLM inference in C/C++.
- [2] MoE-Infinity: Activation-aware expert caching and prefetching for mixture-of-experts inference.
- [3] SpecMD: Speculative mixture-of-experts decoding and least-stale style expert replacement.
- [4] Kwon et al. Efficient memory management for large language model serving with PagedAttention.
- [5] Linux manual pages. `madvise(2)` and `posix_fadvise(2)`.
- [6] Qwen Team. Qwen large language model family and Apache-2.0 model release.